

LONG RANGE CONSTRAINTS FOR NEURAL TEXTURE SYNTHESIS USING SLICED WASSERSTEIN LOSS

Liping Yin^{*,**}, Albert Chua^{**}

CMSE Department, Michigan State University*,
Department of Mathematics, Michigan State University**

ABSTRACT

In the past decade, exemplar-based texture synthesis algorithms have seen strong gains in performance by matching statistics of deep convolutional neural networks. However, these algorithms require regularization terms or user-added spatial tags to capture long-range constraints in images. Having access to a user-added spatial tag for all situations is not always feasible, and regularization terms can be difficult to tune. Thus, we propose a new set of statistics for texture synthesis based on Sliced Wasserstein Loss, create a multi-scale method to synthesize textures without a user-added spatial tag, study the ability of our proposed method to capture long-range constraints, and compare our results to other optimization-based, single-texture synthesis algorithms.

Index Terms— Neural Texture Synthesis, Sliced Wasserstein Distance, Convolutional Neural Networks

1. INTRODUCTION

1.1. Background on texture synthesis

Texture synthesis has been of interest since [1], which was one of the pioneering papers in the field. The objective is to take a reference texture and utilize statistical information from the reference to generate a different texture with similar perceptual properties. While texture synthesis is inherently interesting as a computer vision task, it also provides valuable insight into how humans perceive texture.

Until the emergence of deep convolutional neural networks for image classification, many methods, like [2, 3], have used statistical information of wavelet coefficients for texture generation. However, these approaches failed to produce faithful synthesis for images with complex structures, which suggested more statistical information was needed to capture the essence of a texture. Neural texture synthesis, which generates textures by matching distributions between activation maps of deep convolutional neural networks, started with the seminal work of Gatys et. al. in [6, 7]. The authors used the mean squared error between gram matrices of VGG19 feature maps as a loss function. While the results in [7] were state-of-the-art compared to previous work at the time of publication, the proposed method has trouble capturing long-range constraints (e.g. non-local correlations that influence the texture over larger areas such as the alignment of a brick wall).

Numerous papers have imposed long-range constraints to synthesized images for single texture synthesis [8, 9, 10, 11, 12], which generates one texture per synthesis process. However, these methods

generally require tunable regularization parameters, do not fully capture long-range constraints, or require multiple hours to synthesize one image. Other approaches include universal texture synthesis algorithms [16, 17, 18], which train on a database of textures to reduce synthesis time after a long training period; nonetheless, the focus of this paper will be on single-texture synthesis. Lastly, it is noteworthy to mention methods of capturing long-range statistics from neural networks, such as [24, 25], which could be future avenues to explore.

1.2. Background on Sliced Wasserstein Loss

The authors of [15] propose using Sliced Wasserstein Loss instead of the MSE between gram matrices. The general idea for texture-synthesis is to start with a Gaussian white noise and reference texture. Then, the loss Eq. (2) matches deep feature distributions between the white noise and reference texture, and the white noise is iteratively updated via backpropagation for multiple iterations.

Suppose that layer ℓ of an L layer convolutional neural network has N_ℓ channels and M_ℓ pixels in each channel. We denote the feature vector located at pixel m as $F_m^\ell \in \mathbb{R}^{N_\ell}$. With respect a network architecture, let p^ℓ and $\hat{p}^\ell$ be the probability density functions associated with the set of feature vectors $\{F_m^\ell\}$ and $\{\hat{F}_m^\ell\}$ for images I_1 and I_2 . We assume that the probability density functions are always an average of Dirac delta distributions of the form

$$p^\ell(x) = \frac{1}{M_\ell} \sum_{m=1}^{M_\ell} \delta_{F_m^\ell}(x). \quad (1)$$

The Sliced Wasserstein Loss between two images, $\{w_\ell\}$ are weight terms, typically set to 1 or 0, is given by

$$\mathcal{L}_{\text{SW}}(I_1, I_2) = \sum_{\ell=1}^L w_\ell \mathcal{L}_{\text{SW},\ell}(p^\ell, \hat{p}^\ell), \quad (2)$$

where the Sliced Wasserstein Distance between two feature distributions is given by

$$\mathcal{L}_{\text{SW},\ell}(p^\ell, \hat{p}^\ell) = \mathbb{E}_V[\mathcal{L}_{\text{SWID}}(p_V^\ell, \hat{p}_V^\ell)]. \quad (3)$$

Let $V \in \mathbb{S}^{N_\ell}$ be a random direction on the unit sphere of dimension N_ℓ . Here, we define (with corresponding definitions for $\hat{p}^\ell$)

$$p_V^\ell := \{\langle F_m^\ell, V \rangle\} \quad (4)$$

as a set consisting of batched projections of the feature maps F_m^ℓ onto V ; define vector P_V^ℓ consisting of the elements of p_V^ℓ , the 1D Sliced Wasserstein Loss is the 2-norm between sets of sorted projections:

$$\mathcal{L}_{\text{SWID}}(p_V^\ell, \hat{p}_V^\ell) = \frac{1}{\text{len}(P_V^\ell)} \left\| \text{sort}(P_V^\ell) - \text{sort}(\hat{P}_V^\ell) \right\|_2^2 \quad (5)$$

<https://github.com/liping1005/LongRangeSlicedWasserstein> for code and textures used in experiments. More examples of synthesis provided as well.

We would like to thank Dr. Matthew Hirn for helpful comments and suggestions.

This work was supported in part through computational resources and services provided by the Institute for Cyber-Enabled Research at Michigan State University.

Pitie et al. [4] showed that Sliced Wasserstein Distance satisfies:

$$\mathcal{L}_{\text{SW}}(p, \hat{p}) = 0 \implies p = \hat{p}.$$

The same does not hold for other losses used for texture synthesis, such as the Gram Matrix Loss. Thus, using a Sliced Wasserstein-based loss captures more relevant statistics compared to the traditional Gram Matrix Loss.

While theoretically justified, the method proposed in [15] has limited effectiveness in capturing long-range constraints. Particularly, [15] propose using a spatial tag, or a user-added mask, to constrain the synthesis of textures, but this requires additional human supervision for the creation of the mask. In this work, we propose a simple set of statistics for texture synthesis by adding an additional loss term to the original formulation of Sliced Wasserstein Loss, enabling the modeling of long-range constraints. The proposed set of statistics achieves competitive performance relative to other single-texture synthesis algorithms. Additionally, we augment our synthesis results via a coarse-to-fine multi-scale procedure.

2. PROPOSED TEXTURE SYNTHESIS ALGORITHM

Our proposed method is equivalent to adding a loss term incorporating the Sliced Wasserstein loss across the height dimension in addition to the channel dimension. Due to the locality of the convolution operator, considering the distribution in (1) ignores the correlation between distant pixels, even if such correlations are crucial for generating textures with long-range structure.

To account for locality in an image, consider a set of feature maps $F^\ell \in \mathbb{R}^{H_\ell \times W_\ell \times N_\ell}$ and a feature vector of shape $W_\ell \times N_\ell$, which we denote by $F_{H,n}^\ell \in \mathbb{R}^{H_\ell}$, where $n \in \{1, \dots, W_\ell \times N_\ell\}$. One interpretation is that we have another set of probability distributions to match, which incorporate locality:

$$p_H^\ell(x) = \frac{1}{W_\ell \times N_\ell} \sum_{n=1}^{W_\ell \times N_\ell} \delta_{F_{H,n}^\ell}(x). \quad (6)$$

Thus, we propose adding an additional loss term to match the two probability distributions (1) and (6) simultaneously. Consider distributions p_H^ℓ and $\hat{p}_H^\ell$ associated with the images I_1 and I_2 , respectively, which are derived from Eq. (6). The corresponding additional loss term is

$$\mathcal{L}_{\text{SW},H}(I_1, I_2) = \sum_{\ell=1}^L w_\ell \mathcal{L}_{\text{SW},\ell}(p_H^\ell, \hat{p}_H^\ell), \quad (7)$$

where the weights $\{w_\ell\}_{\ell=1}^L$ are hyperparameters. Our new loss function is the sum of Eq. (2) and Eq. (7)

$$\mathcal{L}_{\text{Slicing}}(I_1, I_2) = \mathcal{L}_{\text{SW}}(I_1, I_2) + \mathcal{L}_{\text{SW},H}(I_1, I_2). \quad (8)$$

Analogous to Eq. (4), we project $\{F_{H,n}^\ell\}$ onto $V_{H_\ell} \in \mathbb{S}^{H_\ell}$, which yield batch projections

$$p_{V_{H_\ell}}^\ell := \{\langle F_{H,n}^\ell, V_{H_\ell} \rangle\}, \quad (9)$$

(and $\hat{p}_{V_{H_\ell}}^\ell$ analogously) where we take the number of projections to be H_ℓ to calculate Eq. (8).

Refer to Algorithm 1 for the pseudocode. We assume that the goal is to synthesize an image the same size as the reference image without any loss of generality. The settings for the slicing loss are to use the first 12 layers of VGG19 for calculating Eq. (2) and the first two convolutions in each convolution block for calculating Eq. (7); we use the L-BFGS optimizer for optimization.

Algorithm 1: Synthesis Algorithm

- 1: Denote the feature map extraction of image I from VGG19 as $\text{Extract}(I)$.
 - 2: Set I_{WN} as the white noise to be updated by an optimizer via backpropagation and I_{Ref} as the reference texture.
 - 3: **for** $k = 1, \dots, M$ Epochs **do**
 - 4: Calculate $\text{Extract}(I_{\text{WN}})$.
 - 5: Calculate $\text{Extract}(I_{\text{Ref}})$.
 - 6: Calculate $\mathcal{L}_{\text{Slicing}}(I_{\text{WN}}, I_{\text{Ref}})$.
 - 7: Backpropagate and update I_{WN} .
 - 8: **end for**
 - 9: Return updated I_{WN} as synthesized texture.
-

2.1. Comparison with Other Methods

We compare our results with algorithms that have similar runtime and computational complexity. All comparisons for the rest of the paper are using 256×256 images. A comparison with SW Loss¹ [15] (without a spatial tag for fair comparison) and with a Gram Matrix Loss + Spectrum Constraint (a constraint on the fourier spectrum of the generated texture) [10] for generating simple, psuedo-periodic textures² with long-range constraints are given in Fig. 1.

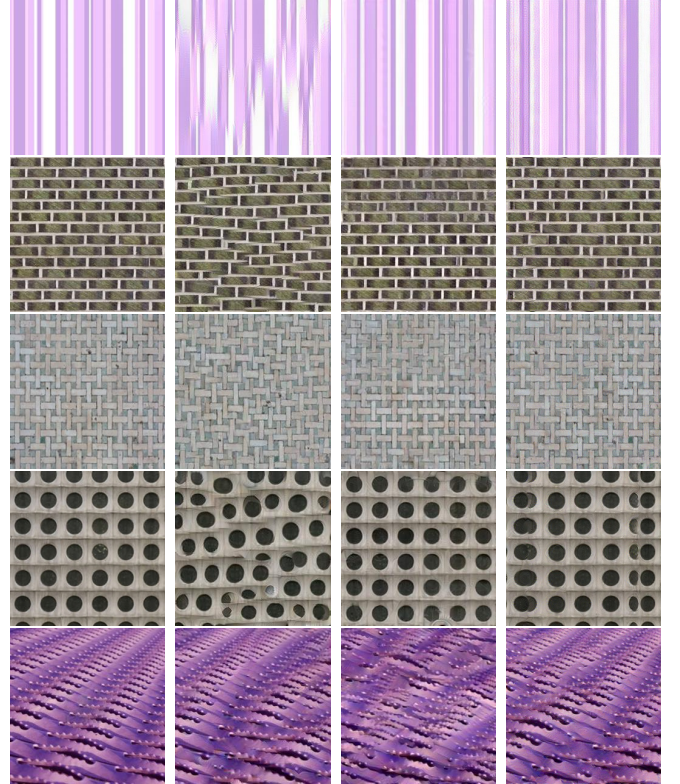


Fig. 1: Comparison of results for pseudoperiodic textures. **Left:** Reference. **Mid Left:** SW Loss. **Mid Right:** Spectrum. **Right:** Using Eq. (8) [Ours].

¹We use the author's TensorFlow implementation, which is a previous commit in <https://github.com/tchambon/A-Sliced-Wasserstein-Loss-for-Neural-Texture-Synthesis>.

²For all the experiments in this paper, our texture sources were the following: the DeepCorr Github Page and the DTD texture database.

From the results in Fig. 1, using our proposed algorithm yields long-range constraints, and the results are comparable to [10]. However, note that the synthesis using our proposed algorithm yields less faithful synthesis relative to [10] in some cases, such as the holes not being circular in the fourth, but can also yield better synthesis compared to [10] in other cases, such as better alignment of the texture in the fifth row. One possible reason for the failure of using Eq. (8) is using Eq. (7) captures only horizontal stationary statistics for each tensor of VGG19 feature maps, but not vertical stationary statistics.

However, many textures have nonstationary components, so a comparison with more complex textures is in Fig. 2. From the re-

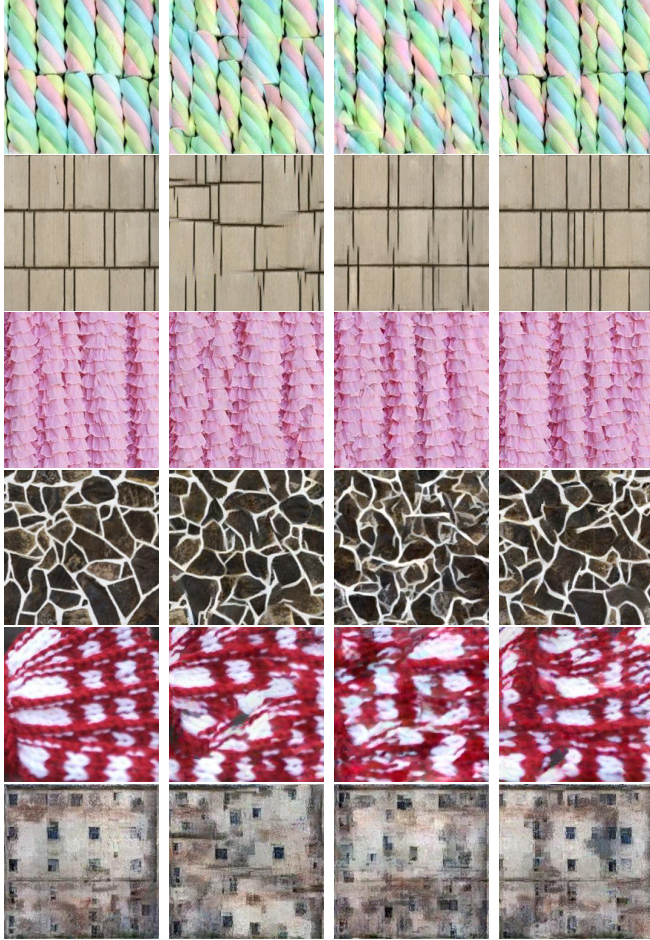


Fig. 2: Comparison of results for more complex textures. **Left:** Reference. **Mid Left:** SW Loss. **Mid Right:** Spectrum. **Right:** Using Eq. (8) [Ours].

sults of Fig. 2, it is clear that our proposed algorithm displays better alignment and overall structure (see the first, third, and fifth row for the most noticeable examples).

A set of 34 textures, which were also used for generating results in this paper, were compiled for a quantitative study between the original SW loss, the spectrum constraint, and our proposed method in Table I. Most of the textures in this set have long range constraints, some pseudo-periodic and some with nonstationary components. In Table I, SW stands for the method using the original SW Loss, Spec. stands for using a spectrum constraint, and GT stands for the Ground

Truth.

We chose the following quantitative metrics: LPIPS [19], FID [20], crop-based FID (this done by taking sixty-four 128×128 crops of the reference texture and synthesized texture for each exemplar. The FID score is calculated between these two sets of images. For the ground truth case, a different set of crops of the reference is used. Five runs are done with 1 SD error bars) similar to [17], KID [21], and crop-based KID (c-KID) score; here the c-KID score is calculated in a similar manner as the c-FID score. For FID and KID based scores, the implementation from [22] is used.

Table I
QUANTITATIVE COMPARISON BY ALGORITHM

Method	LPIPS	FID	c-FID	KID	c-KID
Ours	.44	107.2	72.3 ± 0.34	-.014	$.073 \pm 0.0008$
SW	.45	101.8	78.7 ± 0.48	-.016	$.084 \pm 0.001$
Spec.	.45	99.6	78.3 ± 0.57	-.016	$.085 \pm 0.001$
GT	0	0	18.22 ± 0.17	-0.025	0.0001 ± 0.002

2.2. Ablation Study: Varying the Number of Slices

For the additional loss term in equation Eq. (7), we demonstrate the effect of varying the number of batched projections. Rather than varying the number of slices in each layer by using H_ℓ projections, we use 16, 64, and 256 projections in each layer. The results are presented in Fig. 3. The results of using H_ℓ projections are similar

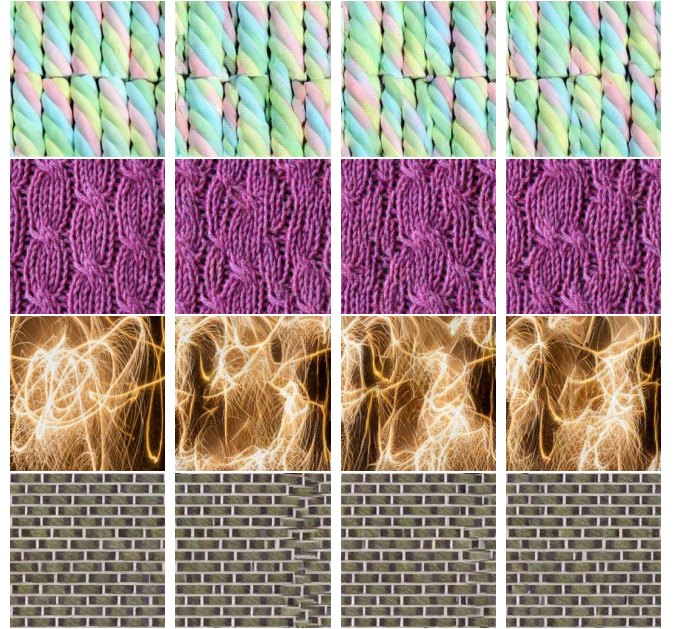


Fig. 3: Textures Generated using a fixed number of directions in each layer for $\mathcal{L}_{SW,H}$. **Left:** Reference. **Mid Left:** 16 directions. **Mid Right:** 64 directions. **Right:** 256 directions.

to the results of using 256 projections in each layer.

3. IMPROVEMENTS VIA A MULTI-SCALE APPROACH

To ameliorate deficiencies in synthesis quality, we propose to augment our synthesis process with a multi-scale procedure from [11,

13, 14] and use our proposed texture synthesis algorithm instead. The general idea is to start from a white noise and reference image both downsampled by 2^K and proceed with Algorithm 1. A texture is then generated and upsampled by a factor of 2. Afterwards, the algorithm is repeated until a texture of the same size is generated. The pseudocode is given in Algorithm 2.

Algorithm 2: Multi-scale Synthesis Algorithm

- 1: Initialize $I_{\text{Synthesis}}$ as a white noise as ref. texture downsampled by 2^K and $I_{\text{ref},i}$ be the ref. texture downsampled by 2^i , and SWSynthesis be synthesis using Algorithm 1.
 - 2: **for** $i = 0, \dots, K$ **do**
 - 3: $I_{\text{Synthesis}} \leftarrow \text{SWSynthesis}(I_{\text{Synthesis}}, I_{\text{ref},K-i})$.
 - 4: $I_{\text{Synthesis}} \leftarrow 2 \times \text{Upsample}(I_{\text{Synthesis}})$.
 - 5: **end for**
 - 6: Return $I_{\text{Synthesis}}$ as the synthesized texture.
-

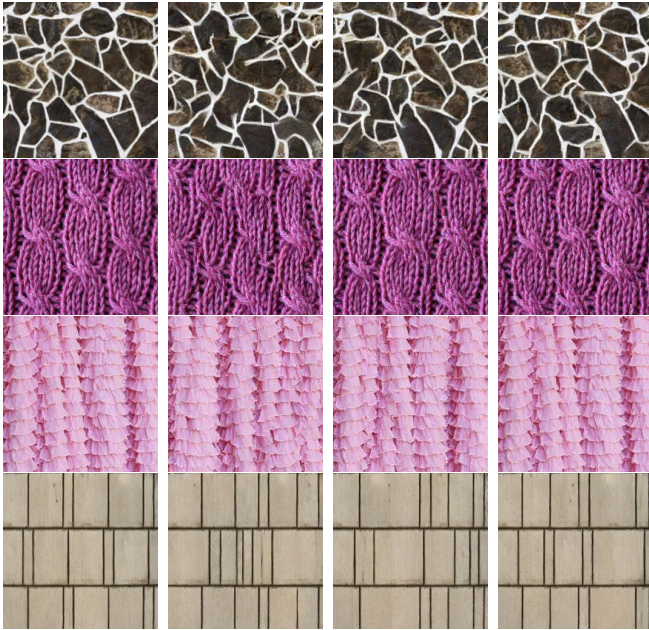


Fig. 4: Multi-scale procedure at different scales. **Left:** Reference. **Mid Left:** $K = 0$. **Mid Right:** $K = 1$. **Right:** $K = 2$.

In Fig. 4, note the small improvements in edge generation and general structure when using $K = 1$ compared to $K = 0$. The details of the texture are generated in a coarse-to-fine way; the initial scale generates the general color and macro-scale features and additional scales add fine-grain details in an image. However, it is possible to create replica textures, which occurred in four cases when $K = 2$. An example is given in Fig. 5. Based on results for generating large textures from [11], we believe the number of scales one can use before generating repetitions depends on the resolution of the image. Our tests found no repetitions when $K = 1$.

In Fig. 6, a small ablation study is done to test the effectiveness of Eq. (8). The multi-scale approach is applied without the additional loss term in Eq. (8), and we see that it is not enough to fully capture nonstationary statistics. Thus, the loss term added in Eq. (8) leads to higher synthesis quality.

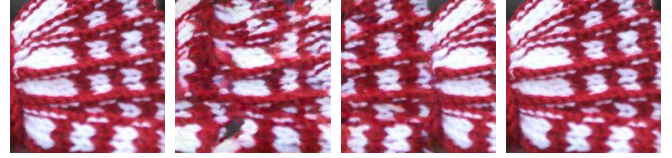


Fig. 5: Progression of synthesis that lead to repetitions. **Left:** Reference Texture. **Middle Left:** $K = 0$. **Middle Right:** $K = 1$. **Right:** $K = 2$.

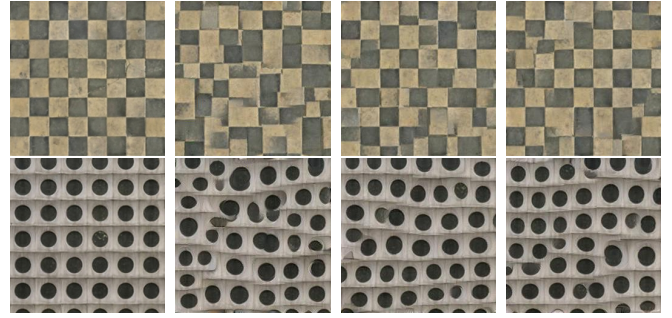


Fig. 6: Ablation study: results with only Eq. (2) **Left:** Reference. **Mid Left:** $K = 0$. **Mid Right:** $K = 1$. **Right:** $K = 2$.

3.1. Additional comparisons

Our proposed algorithm with $K = 1$ is compared with our method with $K = 0$, SW, Gram Matrices + Spectrum Constraint, and the method denoted Gonthier, which uses Gram Matrices + Spectrum Constraint instead of Algorithm 1 in Algorithm 2. In Fig. 6, we provide the results of the synthesis and provide a quantitative study in Table II. Based on these comparisons, we believe our proposed method improves due to the use of Eq. (8) instead a Gram Matrix Loss + Spectrum Constraint or simply using Eq. (2). Note that the perceptual metrics for $K = 2$ will be better than $K = 1$ due to the repetitions, but we provide the results for reference nonetheless.

Table II
COMPARISON OF VARIOUS SYNTHESIS METHODS

Method	LPIPS	FID	c-FID	KID	c-KID
$K = 0$	.44	107.2	72.3 ± 0.34	-.014	$.073 \pm 0.0008$
SW	.45	101.8	78.7 ± 0.48	-.016	$.084 \pm 0.001$
Spec.	.45	99.6	78.3 ± 0.57	-.016	$.085 \pm 0.001$
Gonthier	.42	77.6	68.3 ± 0.35	-.018	$.069 \pm 0.0009$
$K = 1$	.38	67.1	53.5 ± 0.32	-.018	$.043 \pm 0.0006$
$K = 2$	.25	38.3	40.5 ± 0.6	-.022	0.028 ± 0.0009
GT	0	0	18.22 ± 0.17	-.025	0.0001 ± 0.002

4. CONCLUSIONS

We propose a texture synthesis algorithm using a modified version of Sliced Wasserstein Loss that yields a small set of statistics that maintains long-range constraints. Our additional loss term can be considered a regularization term, but without the need for careful hyperparameter tuning. Future work would involve testing the dependence between number of scales and synthesis quality for a large dataset of textures.

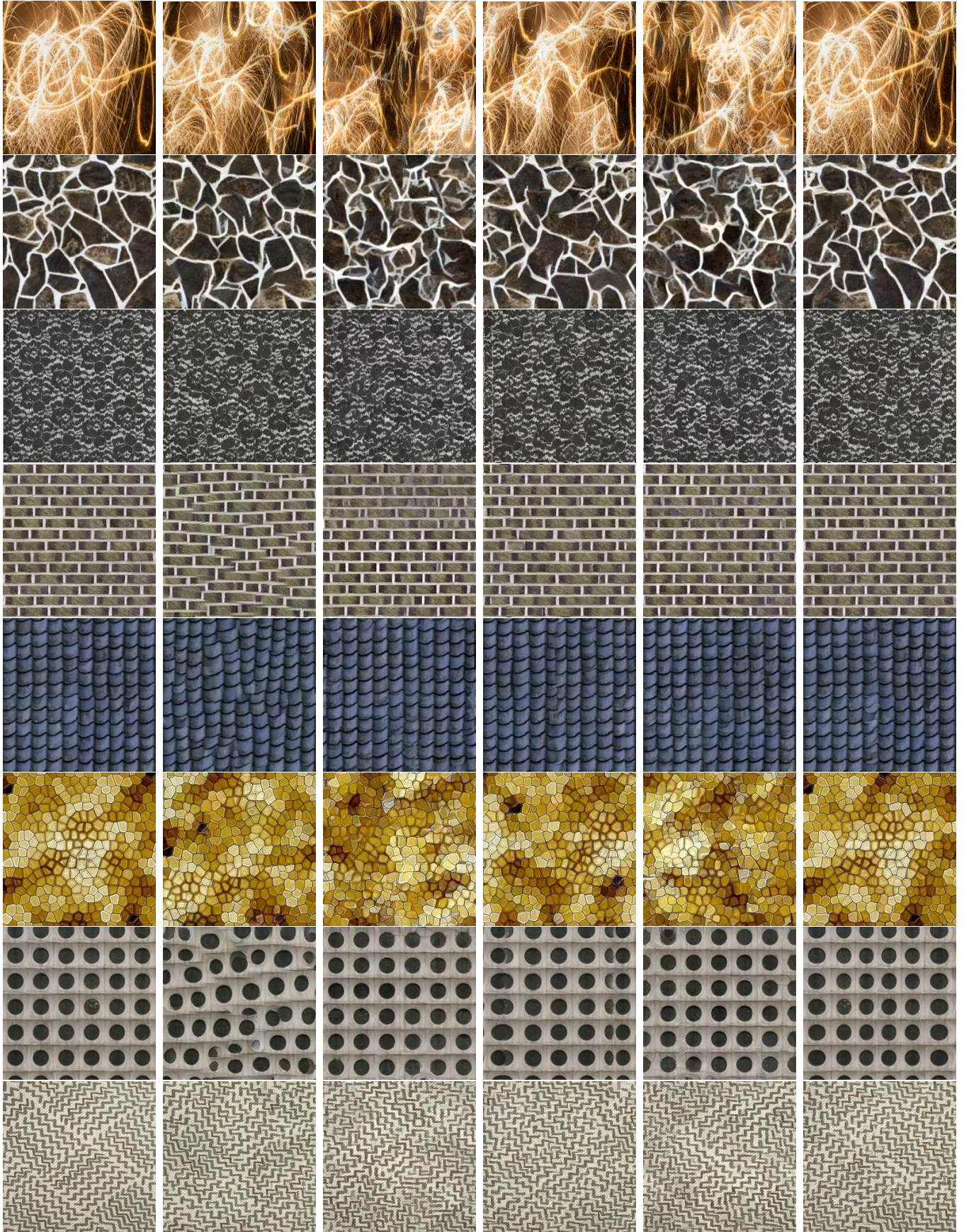


Fig. 7: Additional visual comparison of results. **First Column:** Reference. **Second Column:** SW Loss. **Third Column:** Spectrum Constraint. **Fourth Column:** $K = 0$ (Ours). **Fifth Column:** Gonthier. **Sixth Column:** $K = 1$ (Ours).

5. REFERENCES

- [1] Bela Julesz. “Visual pattern discrimination,” in *IRE Transactions on Information Theory*, vol. 8, no. 2, pp. 84–92, 1962.
- [2] D. J. Heeger and J. R. Bergen. “Pyramid-based texture analysis/synthesis,” in *Proceedings of the 22nd Annual Conference on Computer Graphics and Interactive Techniques*, ACM, 1995, pp. 229–238.
- [3] Javier Portilla and Eero P Simoncelli. “A parametric texture model based on joint statistics of complex wavelet coefficients,” in *International Journal of Computer Vision*, vol. 40, no. 1, pp. 49–70, 2000.
- [4] Pitie, Francois and Kokaram, Anil C and Dahyot, Rozenn “N-dimensional probability density function transfer and its application to color transfer,” in *Tenth IEEE International Conference on Computer Vision (ICCV’05) Volume 1*, 2005.
- [5] Karen Simonyan and Andrew Zisserman. “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *International Conference on Learning Representations (ICLR)*, May 2015.
- [6] Leon Gatys, Alexander S Ecker, and Matthias Bethge. “Image style transfer using convolutional neural networks,” in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2015.
- [7] Leon Gatys, Alexander S Ecker, and Matthias Bethge. “Texture synthesis using convolutional neural networks,” in *Advances in Neural Information Processing Systems* 28, 2015, pp. 262–270.
- [8] Xavier Snelgrove. “High-resolution multi-scale neural texture synthesis,” in *SIGGRAPH Asia 2017 Technical Briefs*, SA ’17, New York, NY, USA, 2017. Association for Computing Machinery.
- [9] Omry Sendik and Daniel Cohen-Or. “Deep correlations for texture synthesis,” in *ACM Trans. Graph.*, 36(4), July 2017.
- [10] Yann Gousseau, Gang Liu, and Gui-Song Xia. “Texture synthesis through convolutional neural networks and spectrum constraints,” in *International Conference on Pattern Recognition (ICPR)*, December 2016.
- [11] Nicolas Gonthier, Yann Gousseau, and Saïd Ladjal. “High resolution neural texture synthesis with long range constraints,” in *Journal of Mathematical Imaging and Vision* 64:478–492, 2022.
- [12] Yang Zhou, Zhen Zhu, Xiang Bai, Dani Lischinski, Daniel Cohen-Or, and Hui Huang. “Non-stationary texture synthesis by adversarial expansion,” in *ACM Transactions on Graphics (ToG)*, vol. 37, ACM, 2018, pp.1-13, 2018.
- [13] V. Kwatra, I. Essa, A. Bobick, and N. Kwatra. “Wasserstein loss for image synthesis and restoration,” in *ACM Transactions on Graphics (ToG)*, vol. 24, ACM, 2005, pp. 795–802.
- [14] Guillaume Tartavel, Gabriel Peyre, and Yann Gousseau, “Variational Texture Synthesis with Sparsity and Spectrum Constraints,” in *Journal of Mathematical Imaging and Vision*, 52:124–144, 2015.
- [15] Eric Heitz, Kenneth Vanhoey, Thomas Chambon, and Laurent Belcour. ‘A Sliced Wasserstein Loss for Neural Texture Synthesis,’ in *In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2021.
- [16] Dmitry Ulyanov, Andrea Vedaldi, and Victor S. Lempitsky. ‘Improved Texture Networks: Maximizing Quality and Diversity in Feed-forward Stylization and Texture Synthesis,’ in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, April 2019.
- [17] Guilin Liu, Rohan Taori, Ting-Chun Wang, Zhiding Yu, Shiqiu Liu, Fitsum A. Reda, Karan Sapra, Andrew Tao, and Bryan Catanzaro ‘Transposer: Universal Texture Synthesis Using Feature Maps as Transposed Convolution Filter,’ in *arXiv:2007.07243 [cs.CV]*, July 2020.
- [18] Morteza Mardani, Guilin Liu, Aysegul Dundar, Shiqiu Liu, Andrew Tao, and Bryan Catanzaro. ‘Neural FFTs for Universal Texture Image Synthesis,’ in *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS)*, April 2020.
- [19] Zhang, Richard and Isola, Phillip and Efros, Alexei A and Shechtman, Eli and Wang, Oliver ‘The unreasonable effectiveness of deep features as a perceptual metric,’ in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2018.
- [20] Heusel, Martin and Ramsauer, Hubert and Unterthiner, Thomas and Nessler, Bernhard and Hochreiter, Sepp ‘Gans trained by a two time-scale update rule converge to a local nash equilibrium,’ in *Advances in neural information processing systems (NIPS)*, 2017.
- [21] Bińkowski, M and Sutherland, DJ and Arbel, M and Gretton, A ‘Demystifying MMD GANs,’ in *International Conference on Learning Representations (ICLR)*, 2018.
- [22] Parmar, Gaurav and Zhang, Richard and Zhu, Jun-Yan ‘On Aliased Resizing and Surprising Subtleties in GAN Evaluation,’ in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2022.
- [23] Chong, Min Jin and Forsyth, David ‘Effectively unbiased fid and inception score and where to find them,’ in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2020.
- [24] Wang, Xiaolong and Girshick, Ross and Gupta, Abhinav and He, Kaiming ‘Non-local neural networks,’ in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2018.
- [25] Zhang, Qingchao and Alvandipour, Mehrdad and Xia, Wenjun and Zhang, Yi and Ye, Xiaojing and Chen, Yunmei ‘Provably convergent learned inexact descent algorithm for low-dose ct reconstruction,’ in *Journal of Scientific Computing*, vol. 101, no. 1, p. 10, 2024.